

Practical 5

AIM: Implementation of a knapsack problem using dynamic programming.

ALGORITHM:

Algorithm:
Input: number of items n , weights $w[i]$, values $v[i]$, capacity w
Output: Maximum Value and selected items
Step1: start
Step2: Read $n, w, w[i]$ and $v[i]$
Step3: initialise of $dp[i][w] = 0$ and $dp[0][j] = 0$
Step4: For $i = 1$ to n , do
Step5: for $j = 0$ to w , do
Step6: if $w[i] \leq j$, then do
 $dp[i][j] = \max(v[i] + dp[i-1][j-w[i]], dp[i-1][j])$
Step7: otherwise do $dp[i][j] = dp[i-1][j]$
Step8: End the inner loop
Step9: End the outer loop
Step10: set $max = dp[n][w]$
Step11: set $j = w$ and trace $i = n$ to 1
Step12: if $dp[i][j] \neq dp[i-1][j]$ then item i is taken
and $j = j - w[i]$
Step13: otherwise, item is not taken
Step14: Display maximum value & selected items
Step15: stop

PROGRAM:

```
#include <iostream> using
namespace std; class
Knapsack
{
private:
    int w[10];    // weights    int
v[10];    // values    int n;    //
number of items    int W;    //
knapsack capacity    int dp[10][100];
// DP table

public:

    // Get Input
void getInput()
{
    cout << "Enter number of items : ";
cin >> n;

    cout << "Enter knapsack capacity : ";
cin >> W;

    for (int i = 1; i <= n; i++)
    {
```

```
cout << "Item " << i << " weight and value : ";

cin >> w[i] >> v[i];

    }

}

// Build DP table

void solve()

{
    // Base case    for
    (int i = 0; i <= n; i++)
        dp[i][0] = 0;

    for (int j = 0; j <= W; j++)
        dp[0][j] = 0;

    // Fill DP table    for
    (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= W; j++)
        {
            if
            (w[i] <= j)
            {
                // Take or don't take
                dp[i][j] = max(                v[i] +
```

```
dp[i - 1][j - w[i]],          dp[i
- 1][j
    );
    }
else
    {
        // Item cannot be taken
dp[i][j] = dp[i - 1][j];
    }
    }
    }
}
```

```
// Print DP table
```

```
void printTable()
{
    cout << "\nDP Table:\n";

    cout << "    ";

    for (int j = 0; j <= W; j++)
        cout << j << "    ";

    cout << endl;
```

```
for (int i = 0; i <= n; i++)
{
    cout << "i=" << i << " ";

    for (int j = 0; j <= W; j++)
cout << dp[i][j] << " ";

    cout << endl;
}
}
```

// Find selected items void

findItems()

```
{
    cout << "\nSelected Items:\n";

    int j = W;

    for (int i = n; i >= 1; i--)
    {
        if (dp[i][j] != dp[i - 1][j])
        {
```

```
        cout << "Item " << i
<< " (w=" << w[i]
        << ", v=" << v[i]
        << ") TAKEN" << endl;
        j = j -
w[i];    }

    else
    {
        cout << "Item " << i
<< " (w=" << w[i]
        << ", v=" << v[i]
        << ") NOT taken" << endl;
    }
}

}

}

// Show result

void showResult()
{
    cout << "\nMaximum Value = " << dp[n][W] << endl;
}

};

int main()
```

```
{  
  
    Knapsack k;  
  
    k.getInput();  
    k.solve();  
    k.printTable();  
    k.showResult();  
    k.findItems();  
  
    return 0;  
}
```

OUTPUT:

```
Enter number of items : 3
Enter knapsack capacity : 2
Item 1 weight and value : 0
1
Item 2 weight and value : 2
2
Item 3 weight and value : 1
2
```

DP Table:

	0	1	2
i=0	0	0	0
i=1	0	1	1
i=2	0	1	2
i=3	0	2	3

Maximum Value = 3

Selected Items:

```
Item 3 (w=1, v=2) TAKEN
Item 2 (w=2, v=2) NOT taken
Item 1 (w=0, v=1) TAKEN
```

TIME COMPLEXITY:

Best case:

```
for i=1 to n
  for j=1 to w
    dp[i][j]
T(n) = n * w
      = nw
T(n) = O(nw)
```

Average case:

```
for i=1 to n
  for j=1 to w
    if (w[i] <= j)
      dp[i][j] = max(c)
    else
      dp[i][j] = dp[i-1][j]
T(n) = n * w
      = nw
T(n) = O(nw)
```

Worst case:

```
for i=1 to n
  for j=1 to w
    dp[i][j] = max(c)
T(n) = n * w
      = nw
T(n) = O(nw)
```

CONCLUSION:

The implementation and time analysis of 0/1 Knapsack using Dynamic Programming helps us understand its efficiency. The algorithm builds a DP table and considers whether to include or exclude each item to find the maximum value within the given capacity. It has a time complexity of $O(nw)$ in the best, average, and worst cases. Therefore, Dynamic Programming provides an efficient and consistent solution for solving the 0/1 Knapsack problem.